Hands-On Machine Learning

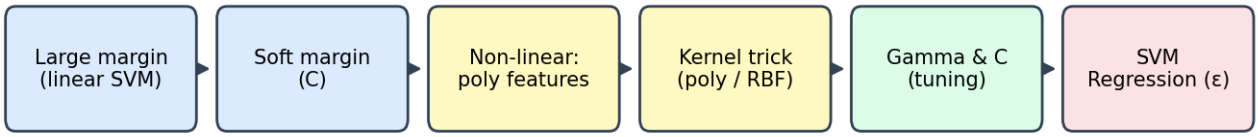
Chapter 5 — Support Vector Machines

Margin · Kernel Trick · RBF · SVM Regression

Summary of your notebook + the missing points filled in + figures and code for revision

From your notebook **Added to fill gaps** **Note / correction**

All numbers in the green **Output (real run)** boxes came from actually running the code (real Iris data, and moons data generated with the same settings as the book). Figures marked (synthetic) use generated data for illustration only.



Chapter roadmap: each step solves a problem left by the previous one.

1 What is an SVM?

From your notebook

What is an SVM? An SVM model can be used for:

- **Classification:** deciding which class a sample belongs to.
- **Regression:** predicting a numeric value.
- **Novelty Detection:** detecting an unusual sample (Ch 9).

← The book notes that SVMs are **especially useful with small and medium-sized Datasets**, particularly for non-linear classification problems, but they **do not scale well** to huge numbers of samples in some of their implementations.

Addition

We will see the reasons for this in numbers in section 5 (the complexity table): the training time of the original `SVC` grows much faster than the number of samples. For very large datasets the book suggests alternatives: `LinearSVC` (linear only) or `SGDClassifier`.

A point we need later: an SVM is originally a **binary classifier**. If you have more than two classes (Multiclass), scikit-learn converts it automatically (OvR or OvO), as we saw in Chapter 3: `SVC` uses OvO and `LinearSVC` uses OvR.

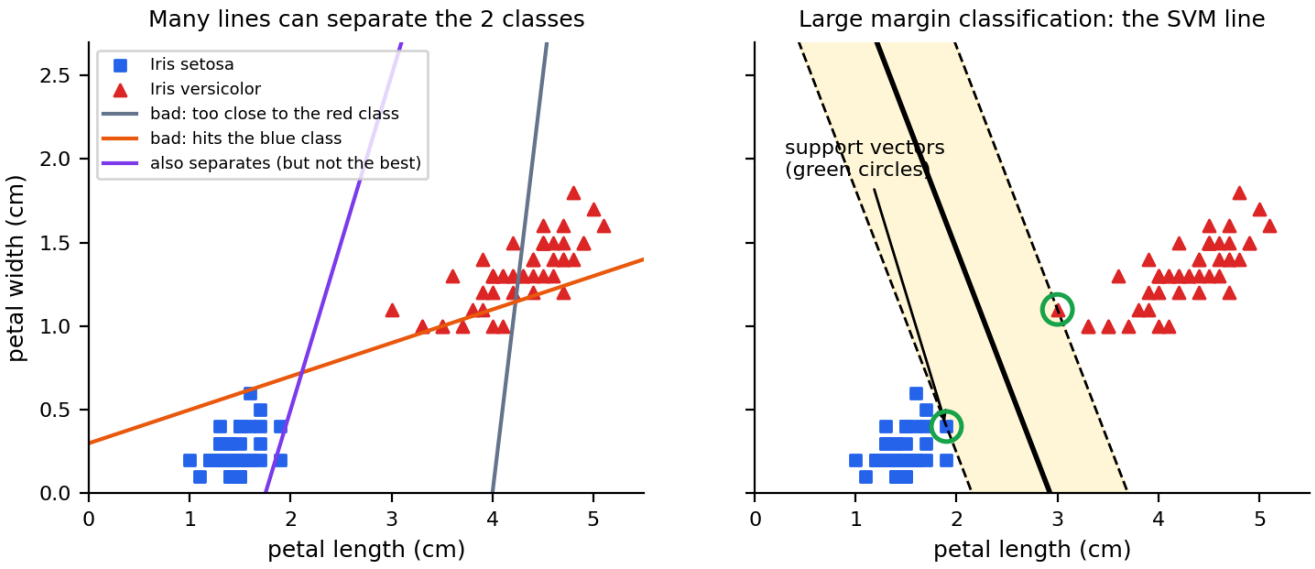
2 Linear SVM: Large Margin Classification

From your notebook

Large Margin classification: imagine you have a Dataset with two classes and you want to separate them with a straight line. You may find more than one line that separates the classes correctly, but not all lines are equally good. For example, a line passing very close to some points: a small change in the data could make it classify new samples badly.

Here the SVM has a different idea: it is not enough for the line to separate the classes, we want the **largest possible distance** between the decision boundary and the nearest points of both classes.

← The space between the two parallel lines around the decision boundary is called the Margin, and this approach is called Large Margin classification.



Real Iris data (setosa vs versicolor). Left: many lines separate the two classes. Right: the SVM picks the line that makes the widest street (yellow). Green circles = the support vectors.

From your notebook

This means choosing the decision boundary that gives the **largest possible Margin**, while respecting the model's constraints. The Support vectors are the training points lying on the edges of the Margin, or those that influence it.

The key idea: the decision boundary is determined essentially by these points; adding other points far from the Margin normally does not change the decision boundary of the solution.

So:

- **Decision Boundary:** the boundary that separates the two classes.
- **Margin:** the space around the decision boundary.
- **Support Vectors:** the points that determine the Margin boundaries. ⇒ and this is the essence of the name Support Vector Machine.

Addition

The idea in equations (from the book): the model computes a score for every sample, and the decision boundary is where the score = 0, while the margin is bounded by score = ±1:

$$\text{score} = \mathbf{w}^T\mathbf{x} + b\ , \qquad \hat{y} = 1 \text{ if score} \geq 0, \quad 0 \text{ otherwise}$$

$$\text{margin width} = \frac{2}{\|\mathbf{w}\|}$$

Addition

This means **maximizing the margin = minimizing ‖w‖**. This is exactly what training does: it looks for the smallest ‖w‖ provided that all points stay outside the street (hard margin, section 4).

And with real numbers on Iris: after training we got `w = [1.294, 0.824]` and `b = -3.788`, so the margin width = 2 / ‖w‖ = **1.304** cm, and the number of support vectors = **2** only (one from each class) out of 100 points!

From your notebook

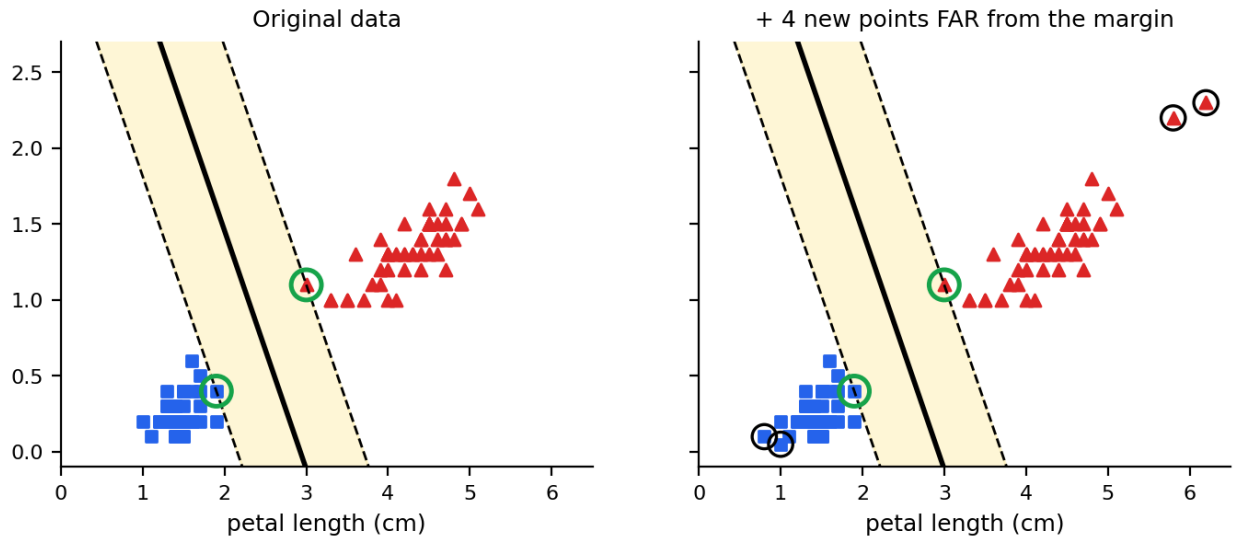
A remark from your notebook (I actually tried it): adding points far from the margin does not change the decision boundary.

```
svm = SVC(kernel="linear", C=1e9).fit(X, y)                      # original data
svm_far = SVC(kernel="linear", C=1e9).fit(X_with_4_far_points, y_with_4)
```

```
print(svm.coef_, svm.intercept_)
print(svm_far.coef_, svm_far.intercept_)
```

Output (real run)

```
[[1.294 0.824]] [-3.7882]
[[1.294 0.824]] [-3.7882]      # identical!
```



Real experiment: we added 4 points (black circles) far from the margin, and the line and margin did not change at all (same `w` and `b`). The support vectors (green circles) are what decide.

Addition

Important remark: this is true for points far from the margin. But a new point **on** the margin or **inside** it (or on the wrong side) becomes a new support vector and changes the solution. That is what happens in the soft margin (section 4).

3 Why do we need Feature Scaling?

From your notebook

An SVM is **very sensitive** to differences in feature scales. ← A feature with a large scale can strongly affect the computation of distances and the Margin, so the result changes depending on the measurement units themselves. That is why we usually apply Scaling before training an SVM.

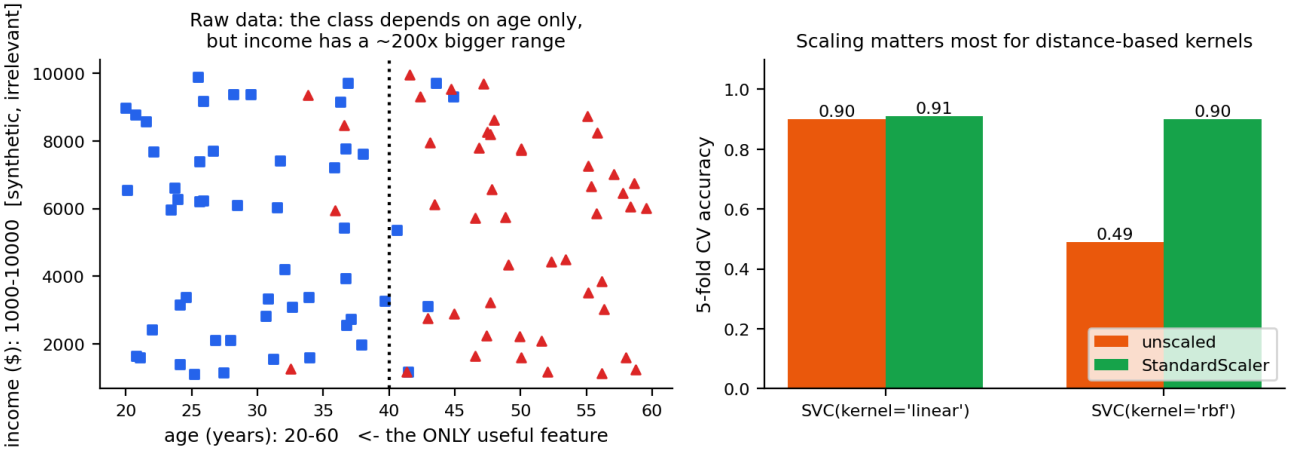
```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score

X_scaled = StandardScaler().fit_transform(X)      # X = [age (20-60), income (1000-10000)]
for kernel in ["linear", "rbf"]:
```

```
    print(kernel, cross_val_score(SVC(kernel=kernel), X, y, cv=5).mean(),
                  cross_val_score(SVC(kernel=kernel), X_scaled, y, cv=5).mean())
```

Output (real run)

```
linear 0.90 0.91      # unscaled  scaled
rbf    0.49 0.90
```



Synthetic data built on purpose: the class depends on `age` only, and `income` is an unrelated column whose range is 200 times larger. The RBF SVM without scaling gets 49% accuracy (a coin flip!), and with scaling 90%.

Addition

Note this detail (it came out of the experiment): the effect of scaling on the **RBF** (and polynomial) was huge (from 0.49 to 0.90) because the whole kernel is built on **distance**, so the large feature swallows the rest. But for the **linear** kernel the difference was small (0.90 vs 0.91) because a linear SVM can "compensate" with a small weight for the large column. That does not mean we ignore scaling for linear: it still affects the margin and convergence speed, so the safe rule: **always** `StandardScaler` **before any SVM**.

The usual rule: `fit` on the training set only, then `transform` the rest (which `make_pipeline` does automatically).

4 Hard Margin and Soft Margin

From your notebook

Imagine we required every sample to be: **(1)** on the correct side of the decision boundary, **(2)** outside the Margin (with no violations). ⇒ This is called **Hard Margin classification**.

But it has two problems:

1. If the data is not linearly separable, there will be no solution satisfying these conditions.
2. A single **outlier** can change the decision boundary drastically or make separation impossible.

So complete strictness is **not suitable for all real data**.

Addition

Hard margin objective:

$$\min_{\mathbf{w},\, b} \frac{1}{2}\mathbf{w}^T\mathbf{w} \quad \text{s.t.} \quad t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq 1 \quad \forall i$$

Addition

(The equation: minimize ‖w‖ provided that every point with `t(i)=±1` satisfies `score ≥ 1`, i.e. completely outside the street.)

From your notebook

Soft Margin classification: instead of insisting that all points satisfy the conditions, we **allow some violations**: a violation can happen when a sample goes inside the Margin or even lands on the wrong side of the decision boundary.

The goal becomes a **balance** between making the Margin large and reducing the number or size of violations. This is called Soft Margin classification.

Addition

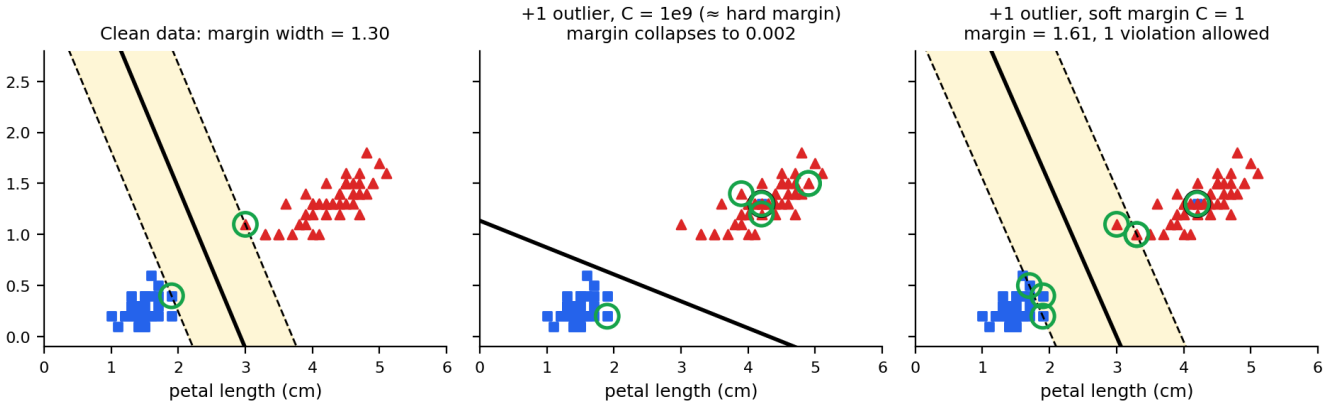
Soft margin objective:

$$\min_{\mathbf{w},\, b,\, \zeta} \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{m}\zeta^{(i)} \quad \text{s.t.} \quad t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq 1 - \zeta^{(i)}, \quad \zeta^{(i)} \geq 0$$

Addition

ζ(i) (zeta) = the violation amount of sample i (zero if there is no violation). And `C` weights the "cost of violations" against the "width of the margin". The same idea in loss form is the **hinge loss**:

$$\text{hinge loss} = \max(0,\ 1 - t \cdot \text{score}) \qquad (t = \pm 1)$$



Real experiment on Iris. Left: clean data, margin = 1.30. Middle: we added one outlier in the middle of the other class (separation is no longer possible): with C=1e9 (≈ hard margin) the margin collapsed to 0.002. Right: the same data with soft margin (C=1): margin = 1.61 and one violation allowed.

From your notebook

The role of the Hyperparameter `C`: in scikit-learn, `C` controls this balance:

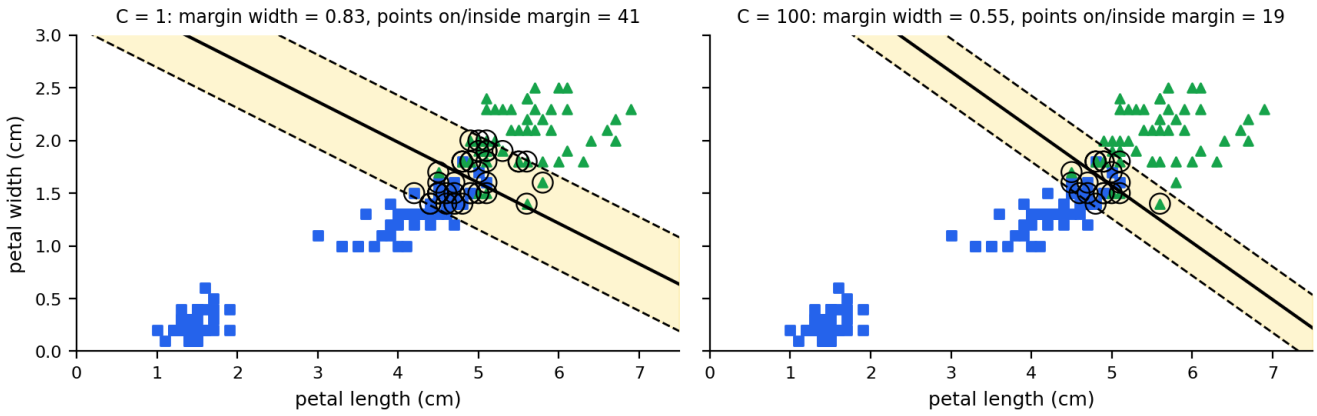
- **Small `C`:** allow more violations and give a larger margin (a wider Margin).
- **Large `C`:** penalize violations more strongly and try to classify the training points more strictly.

⇒ So **reducing `C` gives stronger Regularization**, which can reduce overfitting, but if you reduce it too much the model may underfit.

Remark: a larger `C` does not mean a better model; the right choice depends on the data and the evaluation results.

Iris virginica vs the rest (real data): small C = wide margin, more violations; large C = narrow margin, fewer violations

Iris virginica vs the rest (real data): small C = wide margin, more violations; large C = narrow margin, fewer violations



Real Iris data (virginica vs the rest): C=1: margin wider (0.83) and the number of points on/inside the margin = 41. C=100: margin narrower (0.55) and the points = 19.

Addition

And this confirms your remark with real numbers: the 5-fold cross-validation accuracy of `LinearSVC` on the same data with different `C` values:

C	0.001	0.01	0.1	1	10	100	1000
CV accuracy	0.807	0.867	0.967	0.953	0.953	0.953	0.953

Addition

When `C` is very small (0.001) the model underfits (accuracy 0.81), the best value was `C=0.1` (0.967), and after that the accuracy stays flat. So increasing `C` beyond a certain point does not help. The choice is made with cross-validation (`GridSearchCV`), not by guessing.

5 Linear SVM with scikit-learn

From your notebook

The book uses the Iris Dataset to classify the Iris Virginica flower against the other flowers:

```
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2)    # Iris virginica -> True, any other class -> False

svm_clf = make_pipeline(StandardScaler(), LinearSVC(C=1, random_state=42))
svm_clf.fit(X, y)

X_new = [[5.5, 1.7], [5.0, 1.5]]
print(svm_clf.predict(X_new))
```

From your notebook

`y = (iris.target == 2)` converts the Target into Binary labels: Iris Virginica → True, any other class → False.

Why `random_state=42` ? Some ML algorithms rely on random operations at the start (such as random weight initialization or shuffling the data). Setting a fixed number like `random_state` makes the computer pick random numbers in a fixed, reproducible way, so if you change the value of `C` to see its effect, `random_state` makes sure that any change in the model's accuracy is really caused by changing `C`, not by the computer shuffling the data or starting differently.

The Pipeline: Input Features → StandardScaler → LinearSVC → Prediction.

Output (real run)

```
[True, False]      # train accuracy on Iris = 0.96
```

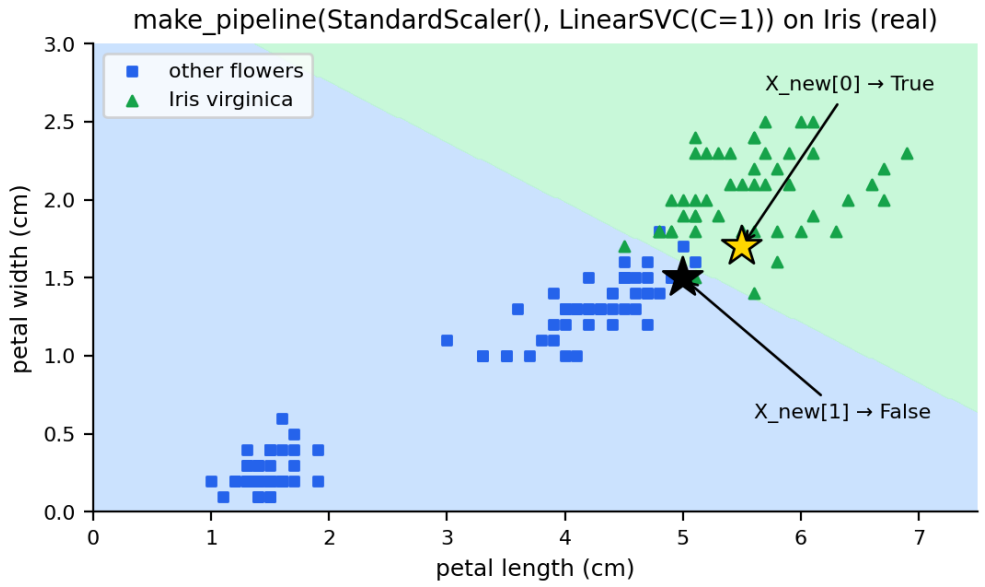
From your notebook

`X_new` here is just numbers: these values were not taken from the Dataset, they are assumed by the book for an experiment to show you how the model works.

Because the model was trained on 2 features written as `[petal length, petal width]`, every new sample fed to it must contain these two values in the same order.

That is: the first flower is Iris Virginica, and the second was classified as `False`.

In real projects `X_new` would be new data that arrived (for example processed images or real sensor measurements) that you want the model to judge although it has never seen them before.



The real decision region of the pipeline on Iris; the two stars = `X_new`.

```
print(svm_clf.decision_function(X_new))
```

Output (real run)

```
[ 0.6616 -0.2204]      # the book prints [ 0.66163411 -0.22036063 ]
```

From your notebook

`svm_clf.decision_function(X_new)` returns **scores** for the sample relative to the decision boundary: e.g. `[0.66, -0.22]` :

- **Positive score** ← the sample is on the Positive side.
- **Negative score** ← the sample is on the Negative side.
- **Score close to zero** ← the sample is close to the decision boundary.

→ The score here is a **measure of the sample's position relative to the decision boundary**, not a Probability.

→ `LinearSVC` has no `predict_proba`, unlike `LogisticRegression`; it does not give a direct probability estimate. The book explains that if you use `SVC(probability=True)` the model adds a process to estimate probabilities from the decision scores using an **extra model**, which increases training cost.

```
svc = make_pipeline(StandardScaler(), SVC(kernel="linear", C=1, probability=True, random_state=42))
svc.fit(X, y)
print(svc.predict_proba(X_new))           # [P(False), P(True)] for each flower
```

Output (real run)

```
[[0.188 0.812]
 [0.637 0.363]]
# hasattr(LinearSVC, 'predict_proba') = False
```

Addition

The result is logical: the first flower (positive score) has a probability of being virginica = **81%**, and the second (negative score) = **36%**. These probabilities are computed by an extra model (internal Platt scaling + cross-validation), not by the SVM itself, so they are an **estimate**, not a fact. (On this small data the time difference is not noticeable: 1.4 ms vs 1.9; but on large data it matters.)

6 Nonlinear SVM Classification

There are 3 ways to train a linear SVM in scikit-learn, and this table is from the book (with a real timing experiment below it):

Class	Time complexity	Out-of-core	Needs scaling?	Kernel trick
LinearSVC	O(m × n)	No	Yes	No
SVC	O(m² × n) to O(m³ × n)	No	Yes	Yes
SGDClassifier	O(m × n)	Yes	Yes	No

(m = number of samples, n = number of features.) And a real timing experiment on moons (seconds, single-core machine):

Number of samples m	LinearSVC	SVC (rbf)	SVC / Linear ratio
500	0.0015	0.0023	2x
2000	0.0015	0.0119	8x
6000	0.0028	0.0773	27x

When m grew 12 times (500 → 6000), the time of LinearSVC grew about 3 times only, but the time of SVC grew about 40 times. This is roughly what the book means by "does not scale well to huge numbers of samples".

From your notebook

What if the data cannot be separated by a straight line? A linear SVM may be unsuitable if the data shape is non-linear. We used to add new Features such as x^2 so the model could represent a curved relationship (Polynomial Regression in Ch 4). The same idea can be used with SVM.

Imagine you have one Feature x and a complex separation relationship; we add a new Feature: $x2 = x1^2$. So the data is now represented by the original Feature and a Feature derived from it.

The general idea: transform the data into more expressive Features, then use a linear SVM on them.

Implementing a linear SVM with Polynomial features

From your notebook

The book uses `make_moons()`, a toy dataset of two interleaving half-moons:

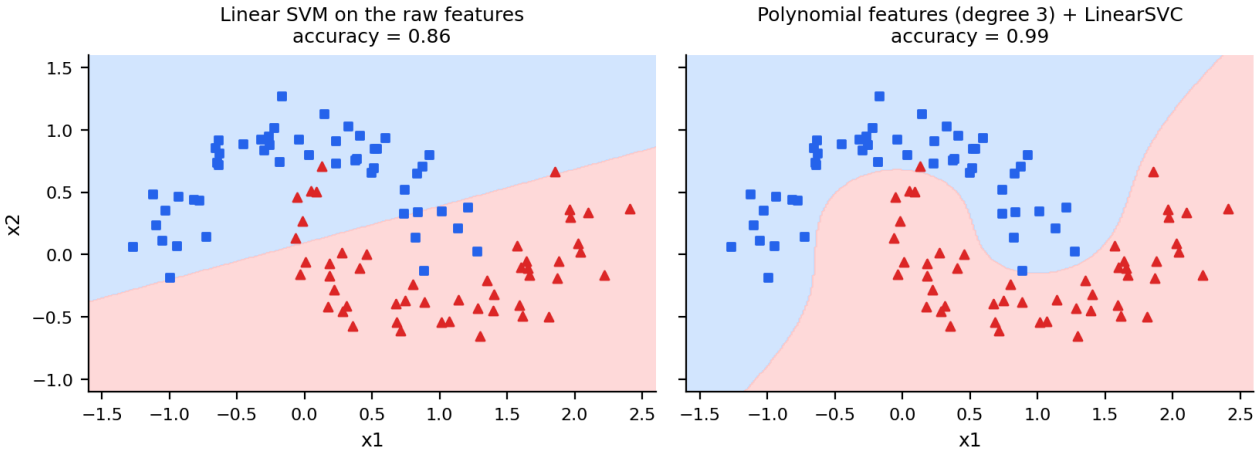
```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import LinearSVC

X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
polynomial_svm_clf = make_pipeline(PolynomialFeatures(degree=3),
                                   StandardScaler(),
                                   LinearSVC(C=10, max_iter=10000, random_state=42))
```

```
polynomial_svm_clf.fit(X, y)
print(polynomial_svm_clf.score(X, y))
```

Output (real run)

```
0.99      # (a plain linear SVM on the raw features: 0.86)
```



The moons data (the two half-moons). Left: linear SVM on the original features (accuracy 0.86). Right: after adding polynomial features of degree 3 the decision boundary became curved (accuracy 0.99).

From your notebook

The Pipeline does: **(1)** add Features up to degree three. **(2)** Scaling for the new Features. **(3)** train a linear SVM on the transformed data.

But there is a problem: as the Degree or the number of original Features grows, the number of new Features can become very large and slows training and needs more memory, and here the Kernel Trick comes in.

`max_iter=10000` : the maximum number of iteration steps the algorithm is allowed. The default is only 1000, but after adding `PolynomialFeatures(degree=3)` the Features became many and complex, which makes the algorithm need more steps to tune the weights and make sure it reached the best convergence.

Note / Correction

Remark from the actual experiment: `max_iter=10000` is **not necessary** in this case: I tried the model and it converged in only **10 iterations** (and when I raised the degree to 10 it needed 31 iterations), so the default (1000) was enough and no ConvergenceWarning appeared. The book includes it as a precaution. The rule: if a `ConvergenceWarning` appears raise `max_iter`, and if it doesn't there is no need.

And regarding the number of features: on this data (2 features, degree 3) we have **10 features** (including the bias). But if you have 100 features and degree 3 it becomes **176,851**, and with degree 10 it becomes **46,897,636,623,981** (47 trillion!). This is exactly why we need the kernel trick.

7 Kernel Trick and Polynomial Kernel

From your notebook

What is the Kernel Trick? Let's build the idea step by step: we want to add new Features that help us separate non-linear data, but sometimes the explicit transformation of the data produces a **huge**, or even an **infinite**, number of dimensions in some transformations.

→ The Kernel Trick lets the SVM obtain a **similar result** to what would be obtained using a given Features transformation, **without us computing the transformation itself explicitly**.

→ The idea: rely on a **function that computes a mathematical relationship between two samples**, instead of creating all the new Features.

Addition

The polynomial kernel:

$$K(\mathbf{a}, \mathbf{b}) = (\gamma\, \mathbf{a}^T \mathbf{b} + r)^d \qquad (d = \text{degree},\ r = \text{coef0})$$

Addition

This is the polynomial kernel: instead of computing `PolynomialFeatures` and multiplying, we compute `(γ·aᵀb + r)^d` directly between the two samples (just `a·b` on the original features). Its result has the same effect as adding all the features up to degree `d`, without building those 47 trillion columns.

From your notebook

Polynomial kernel example:

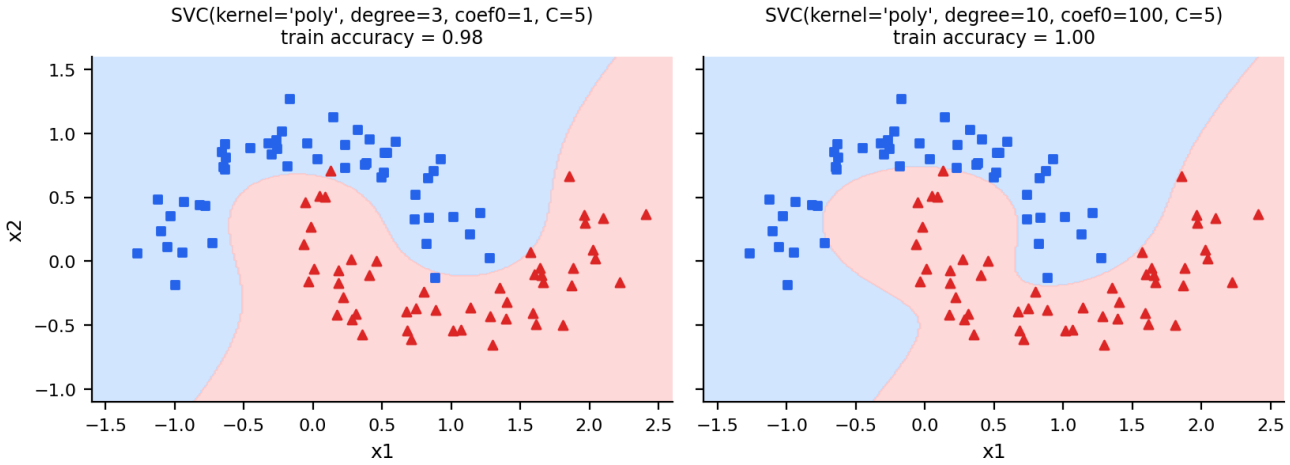
```
from sklearn.svm import SVC
poly_kernel_svm_clf = make_pipeline(StandardScaler(),
                                    SVC(kernel="poly", degree=3, coef0=1, C=5))
poly_kernel_svm_clf.fit(X, y)
print(poly_kernel_svm_clf.score(X, y))
```

Output (real run)

```
0.98      # support vectors = 19 of 100
```

From your notebook

- The Polynomial kernel uses `SVC(kernel="poly", degree=3, coef0=1, C=5)`.
- `coef0` → controls the influence of lower-degree terms compared with higher-degree terms.
- If the model overfits it may be appropriate to **reduce the degree** of the Polynomial kernel, and if it underfits you can try a higher degree, **with evaluation** instead of assuming that increasing always improves performance.



Real run. Left: `degree=3, coef0=1, C=5` (accuracy 0.98, 19 support vectors). Right: `degree=10, coef0=100, C=5` (accuracy 1.00, 8 support vectors): a much more complex curve. A training accuracy of 1.00 here is a warning sign: possible overfitting, and the judgment is by validation, not by training.

8 Gaussian RBF Similarity Features

From your notebook

Similarity Features are a characteristic that expresses how similar a given sample is to something else (like a reference point). Imagine you are building a house-price prediction model and want to add a Feature (not present in the data) that tells you: **how close is this apartment to the city center?**

The idea here: instead of being satisfied with the original Features, we create Features that represent how similar each sample is to reference points called **landmarks**.

→ The **Gaussian RBF Similarity Feature** takes the following form:

$$\phi_\gamma(\mathbf{x}, \ell) = \exp\left(-\gamma \|\mathbf{x} - \ell\|^2\right)$$

From your notebook

(ℓ = the landmark, x = the sample, $\|x - \ell\|^2$ = the squared distance between them, γ controls how fast similarity decays with distance.)

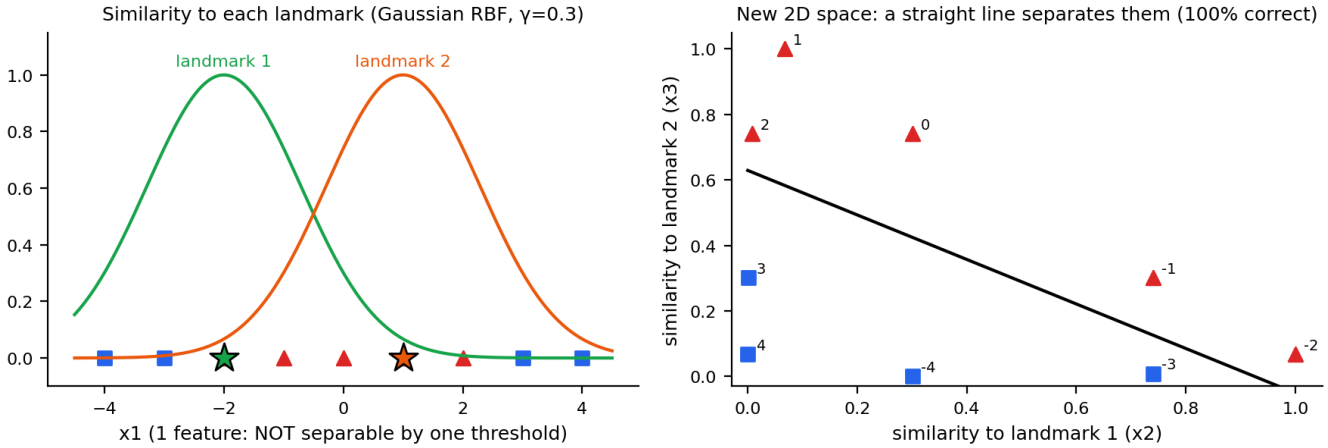
→ The value is **close to 1** when the sample is near the landmark and approaches **zero** as it moves away from it, and thus every sample can get a Similarity Feature for each landmark.

```
import numpy as np
def rbf(x, landmark, gamma): return np.exp(-gamma * (x - landmark) ** 2)

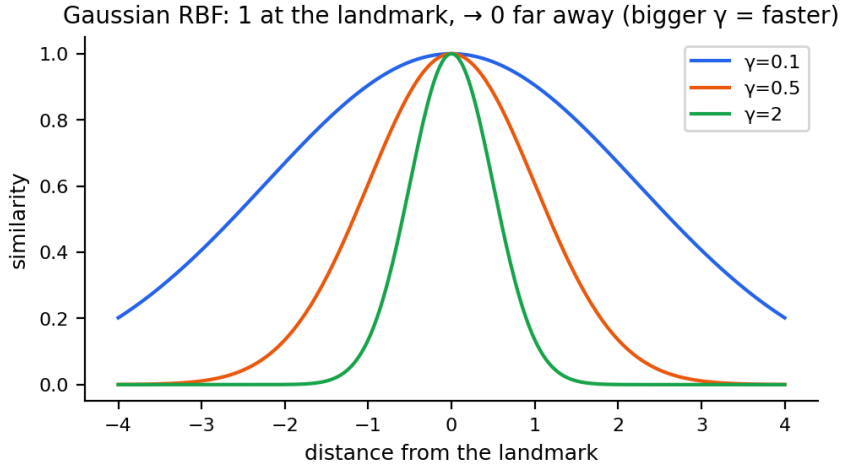
x1 = np.arange(-4, 5)                    # 9 points, 1 feature:  -4 ... 4
y  = [0, 0, 1, 1, 1, 1, 1, 0, 0]         # the middle points are class 1 -> NOT separable by one
threshold
F = np.c_[rbf(x1, -2, 0.3), rbf(x1, 1, 0.3)]     # 2 similarity features (landmarks at -2 and 1)
print(F[2].round(3))                     # the point x1 = -2  (sits exactly on landmark 1)
print(F[3].round(3))                     # the point x1 = -1
```

Output (real run)

```
[1.000 0.067]
[0.741 0.301]
```



The same example as the book, run for real. Left: one dimension, and the red points (the middle) are surrounded by blue ones on both sides, so no single threshold separates them (a one-line accuracy of 0.56). Right: after converting each point into similarity to the two landmarks, it becomes 2D and a straight line separates them 100% correctly.



The RBF curve: 1 at the landmark, and it decreases with distance. A larger γ = narrower and faster drop.

From your notebook

→ **The problem:** creating a Landmark for every sample can greatly increase the number of Features, so if you have `m` samples you may need `m` new Features. The Kernel Trick tries to achieve a **similar effect without explicitly creating all these Features**.

9 Gaussian RBF Kernel and Gamma

From your notebook

The same formula as the (Gaussian) Similarity Feature and almost the same thing, but the difference is that the RBF kernel computes the relationship between two samples using the **distance between them**, and you do not have to specify landmarks manually and create the Similarity Features yourself.

$$K(\mathbf{a}, \mathbf{b}) = \exp\left(-\gamma \|\mathbf{a} - \mathbf{b}\|^2\right)$$

```
rbf_kernel_svm_clf = make_pipeline(StandardScaler(),
                                   SVC(kernel="rbf", gamma=5, C=0.001))
rbf_kernel_svm_clf.fit(X, y)
print(rbf_kernel_svm_clf.score(X, y))
```

Output (real run)

```
0.97
```

From your notebook

→ When we used `kernel="rbf"` it means Gaussian Radial Basis Function.

Addition

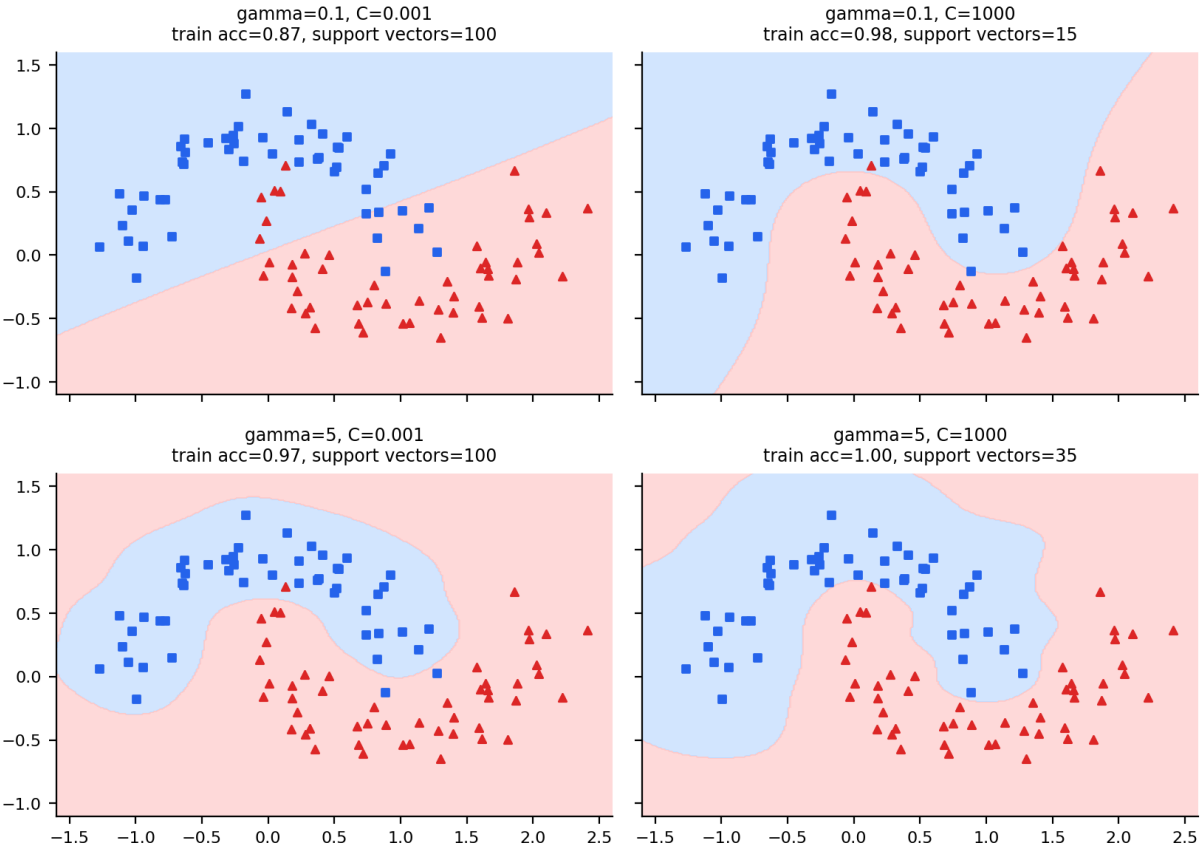
A small clarification of your sentence "no need to specify landmarks": the RBF kernel in practice treats every training point as an implicit landmark (the book says so), but without actually building the `m` columns. And this is the secret of why SVC gets slow when `m` grows (section 5): the kernel is computed between every pair of points.

From your notebook

⇒ **The `gamma` Hyperparameter:**

- If gamma increases: (1) the influence of each sample becomes more local (similarity drops quickly) (2) the decision boundary becomes more wiggly (3) the risk of overfitting increases.
- If gamma decreases: (1) the influence of each sample extends over a larger distance (2) the decision boundary usually becomes smoother (3) if you reduce it more than necessary underfitting may happen.

⇒ So gamma controls the range of a sample's influence on the shape of the decision boundary, and affects model complexity. ⇒ That is why the book considers it similar to a Regularization hyperparameter.

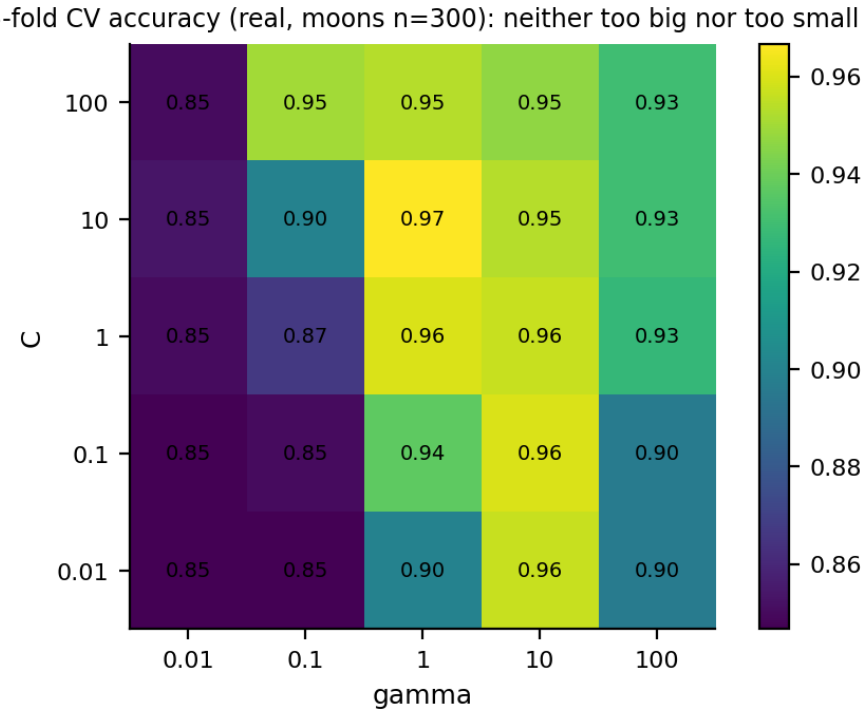


Real run of the effect of `gamma` and `C` on the moons (the same values as in the book). Top (`gamma=0.1`, small): the boundary is smoother. Bottom (`gamma=5`, large): the boundary is complex. And `C=0.001` (left) gives a constrained model (100 support vectors), and `C=1000` (right) tries to fit every point.

gamma	C	Training accuracy	Number of support vectors
0.1	0.001	0.87	100
0.1	1000	0.98	15
5	0.001	0.97	100
5	1000	1.00	35

Addition

Note: `C=0.001` makes **all** the points (100 out of 100) support vectors because the margin is very wide and most points are inside it; and `C=1000` reduces them to 15 or 35. This is consistent with your remark: small `C` = more flexibility = more points inside the margin.



Choosing correctly with cross-validation (5 folds, a larger moons dataset): the best combination was `gamma=1, C=10` with accuracy 0.967. Values that are too small (left/bottom) underfit, and values that are too large lower the accuracy a bit (overfitting).

```
from sklearn.model_selection import GridSearchCV
param_grid = {"svc__C": [0.01, 0.1, 1, 10, 100], "svc__gamma": [0.01, 0.1, 1, 10, 100]}
grid = GridSearchCV(make_pipeline(StandardScaler(), SVC(kernel="rbf")), param_grid, cv=5).fit(X, y)
print(grid.best_params_, round(grid.best_score_, 3))
```

Output (real run)

```
{'svc__C': 10, 'svc__gamma': 1} 0.967
```

10 SVM Regression

From your notebook

Is the SVM only for Classification? No, it can also be used with Regression.

→ Remember in Classification we tried to draw the **widest possible street** between two classes? In SVM Regression the **goal is different:** we try to find a function that fits **as many samples as possible inside a street** whose width is set by the Hyperparameter ε (epsilon), while reducing the penalty for samples outside it.

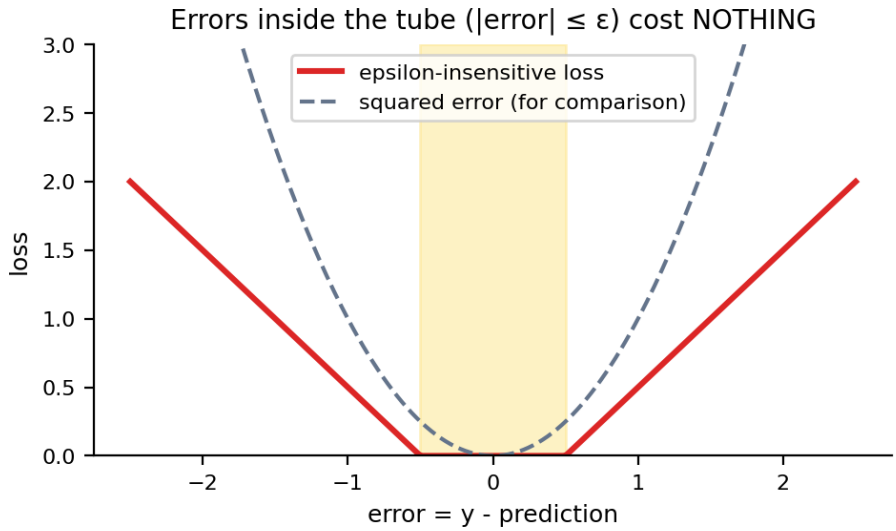
→ If a sample's prediction is **inside the ε range** around the target value, it does not bear a penalty of the Epsilon-Insensitive loss type merely for having an error inside this range.

→ Samples **outside the range** are the ones that contribute to the error according to how far away they are, taking the model settings into account.

Addition

The epsilon-insensitive loss:

$$\ell_\varepsilon(y, \hat{y}) = \max(0,\ |y - \hat{y}| - \varepsilon)$$



The SVR loss: the yellow band is the tube (error inside it = zero). Outside the tube the error is computed linearly (red), unlike the squared error (dashed).

From your notebook

- **Large ε:** a wider tolerance range, so more samples can remain inside the range without penalty.
- **Small ε:** a narrower tolerance range, so more samples become influential in determining the solution.

→ The book indicates: reducing ε increases the number of Support Vectors, and samples that stay inside the Margin (the street) do not influence the predictions the way samples outside it do.

From your notebook

Linear SVM Regression

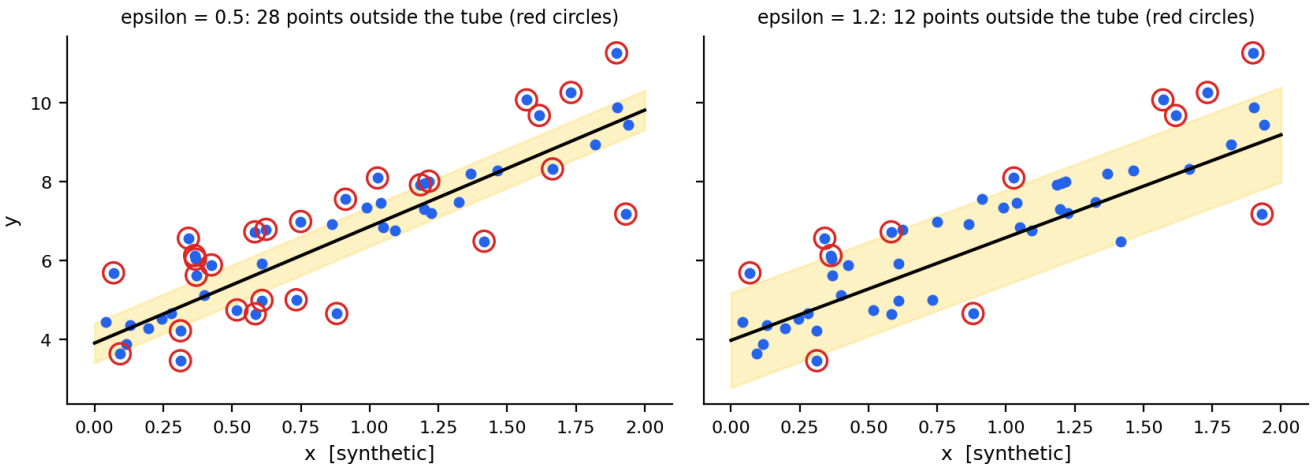
```
from sklearn.svm import LinearSVR
svm_reg = make_pipeline(StandardScaler(), LinearSVR(epsilon=0.5, random_state=42))
svm_reg.fit(X, y)
print(svm_reg.predict([[1.0]]))
```

Output (real run)

```
[6.8566]      # (synthetic data: y = 4 + 3x + noise  ->  about 7 at x = 1)
```

From your notebook

The idea: `Features → StandardScaler → LinearSVR → Predicted Numeric Value`. This is used for Linear SVM Regression.



Real experiment: reducing ε from 1.2 to 0.5 made the tube narrower, so the points outside it (red circles) increased from 12 to 28, which is exactly the meaning of "small ε = more samples influential".

From your notebook

Nonlinear SVM Regression

If the relationship is non-linear, SVR can be used with a kernel:

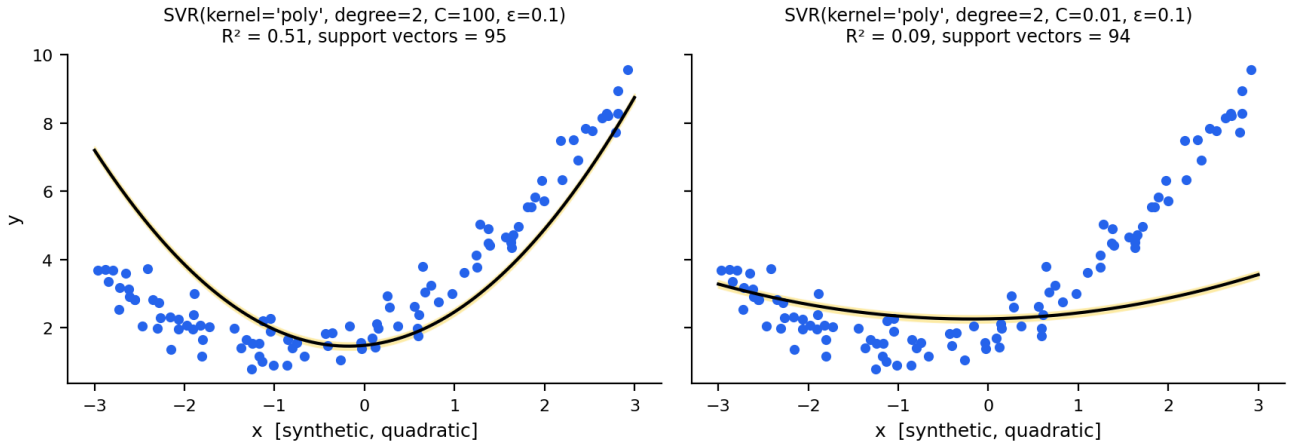
```
from sklearn.svm import SVR
svm_poly_reg = make_pipeline(StandardScaler(),
                             SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1))
svm_poly_reg.fit(X, y)
print(svm_poly_reg.predict([[1.5]]))
```

Output (real run)

```
[2.6203]      # (synthetic quadratic data: y = 0.5x² + x + 2 + noise  ->  about 4.6 at x = 1.5)
```

From your notebook

→ Here `SVR` is the SVM version for Regression that supports the Kernel Trick. → The book explains that `SVR` can become **slow** with a large number of samples, while `LinearSVR` **scales roughly linearly** with the data size.



Quadratic data (synthetic). Left: `C=100`: the SVR follows the curve reasonably. Right: `C=0.01`: the model is very constrained (flat) and R² is only 0.09. Same logic as `C` in classification: small `C` = strong regularization.

Addition

Note the value that came out of the experiment: the prediction at 1.5 = only 2.62, although the real data at 1.5 is about 4.6. The reason is that `C=0.01` makes the model very constrained, and the `coef0` default = 0, so the kernel has no first-degree terms. If we add `coef0=1` (as we learned in section 7) with `C=100`, R² = 0.96 instead of 0.51. So `coef0` and `C` matter for SVR just as they do for SVC.

And a final remark: the SVM for Novelty detection (which you mentioned at the start of the chapter) is called `OneClassSVM` and is explained in Chapter 9.

11 Quick review questions

Answer first in your head, then check the answers on the next page.

1. What is an SVM? And what are its uses (3 uses)?
2. When is an SVM useful? And when is it slow?
3. Explain the idea of Large Margin classification. And how does it differ from any line that separates the classes?
4. What are the Decision Boundary, the Margin and the Support Vectors?
5. What happens if we add points far from the margin? And what if we add a point on the margin?
6. Why is an SVM sensitive to feature scale? And what is the difference in effect between linear and rbf?
7. What is the Hard Margin? And what are its two problems?
8. What is the Soft Margin? And what is ζ?
9. What is the role of `C`? And what happens if we make it smaller or larger?
10. Does a larger `C` mean a better model? And how do we choose it?
11. Why do we use `random_state`?
12. What does `decision_function` mean in an SVM? And is it a probability?
13. Why does `LinearSVC` have no `predict_proba`? And what is the solution?
14. What are the steps of the pipeline LinearSVC + StandardScaler + PolynomialFeatures? And what is its problem when the degree grows?
15. What is the Kernel Trick?
16. What is `coef0` in the polynomial kernel?
17. What are Similarity Features and landmarks? And write the Gaussian RBF equation.
18. What is the difference between RBF similarity features and the RBF kernel?
19. What is the effect of gamma when it gets larger or smaller?
20. Why does the book liken gamma to a regularization hyperparameter?
21. How does SVM Regression work? And what is the role of ε?
22. What happens to the number of support vectors when we reduce ε?
23. What is the difference between LinearSVR and SVR?
24. What is the difference in complexity between LinearSVC, SVC and SGDClassifier?

12 Review question answers

1. Support Vector Machine: a powerful model used for Classification, Regression and Novelty Detection.
2. Useful with small and medium datasets, especially non-linear classification, and slow (does not scale) with a huge number of samples (SVC in particular).
3. Instead of any line that separates the classes, we pick the line that gives the largest margin between it and the nearest points of both classes, so it is less sensitive to a small change in the data.
4. Decision boundary: the separating boundary. Margin: the street around it. Support vectors: the points on the edge of the margin or inside it, which determine the solution.
5. Points far from the margin do not change the solution (we tried it: same w and b). A point on/inside the margin becomes a new support vector and changes the solution.
6. Because the margin and the kernel are built on distances, so the large feature dominates. The effect was huge on rbf (0.90 → 0.49) and small on linear (0.91 → 0.90), but scaling is best always.
7. All points must be outside the margin and on the correct side. Its problems: no solution if the data is not linearly separable, and it is very sensitive to outliers (the margin collapses).
8. We allow some violations (points inside the margin or on the wrong side). ζ = the violation amount of each sample, and the goal is a balance between a large margin and few violations.
9. Small C: more violations + wider margin + stronger regularization (risk of underfitting). Large C: stronger penalty for violations + narrower margin (risk of overfitting).
10. No. The best choice depends on the data and the evaluation results (GridSearchCV / cross-validation).
11. So that randomness (shuffling/initialization) stays fixed, so any change in the result is caused by changing the hyperparameter, not by randomness.
12. score = a measure of the sample's position relative to the decision boundary (positive = Positive, negative = Negative, near 0 = close to the boundary). Not a probability.
13. Because it does not estimate probabilities directly. The solution: SVC(probability=True), which adds an extra model to estimate the probabilities (and increases training time).
14. Add features of degree 3, then scaling, then a linear SVM. Its problem: the number of features explodes with the degree and the number of original features (slowness and memory).
15. A way to obtain a result similar to adding many features (even infinite) without computing them explicitly, using a kernel function that computes the relationship between two samples.
16. It controls the influence of lower-degree terms relative to higher-degree terms. In SVR the experiment showed that coef0=0 deprives the model of first-degree terms.
17. Similarity feature = how similar a sample is to a reference point (landmark): $\exp(-\gamma\|x-\ell\|^2)$, which is 1 at the landmark and decreases with distance.
18. Similarity features: we explicitly build m columns (landmarks). RBF kernel: computes the similarity between two samples directly without building the columns.
19. Large gamma: each sample's influence is local, the boundary is wiggly, risk of overfitting. Small gamma: extended influence, smoother boundary, risk of underfitting.

20. Because gamma controls model complexity (like regularization): smaller gamma = a simpler, smoother model.
21. It tries to fit as many samples as possible inside a street of width ε; errors inside it have no penalty, and the penalty applies only to samples outside, according to their distance.
22. The number of support vectors increases (more points fall outside the tube).
23. LinearSVR is linear and scales roughly linearly with m. SVR supports kernels but becomes slow with large m.
24. LinearSVC: $O(m \times n)$, no kernel trick. SVC: $O(m^2 \times n)$ to $O(m^3 \times n)$ with a kernel. SGDClassifier: $O(m \times n)$ and supports out-of-core.